

FINAL PROJECT REPORT

MLOps Assignment 1

Heart Disease Prediction Pipeline

Student: Rajesh Natarajan

Course: Machine Learning Operations (AIMLCZG523)

Dataset: UCI Cleveland Heart Disease dataset

Repository: <https://github.com/Rajvik85/MLOPS-Assignment-1>

For this assignment, I built and tested a complete machine learning workflow covering data download, EDA, preprocessing, model training, experiment tracking, API serving, automated tests, Docker, and Kubernetes.

This report uses outputs from the repository and screenshots captured while running the project. The final section maps each production-readiness and reporting requirement to the evidence included here.

1. Setup, installation, and architecture

I used the commands below for a clean local setup. The same requirements file is installed by GitHub Actions and copied into the Docker build.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python src/download_data.py
python -m pytest tests/ --cov=src --cov-report=term-missing
```

Architecture diagram

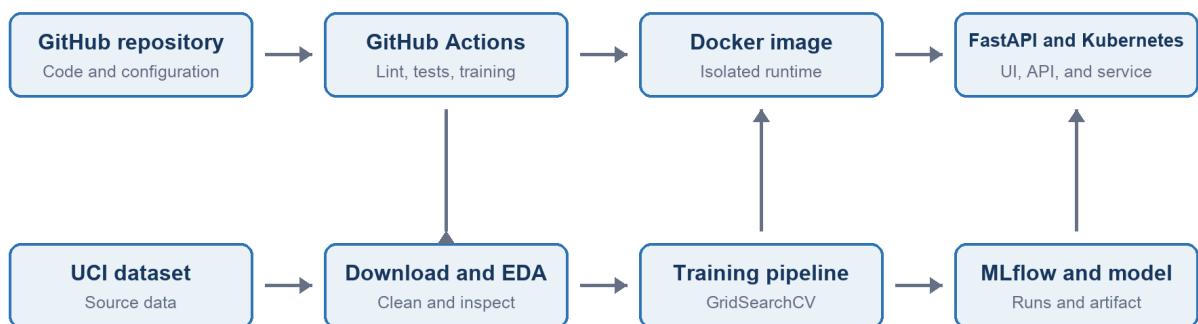


Figure 1. End-to-end project architecture.

Repository structure

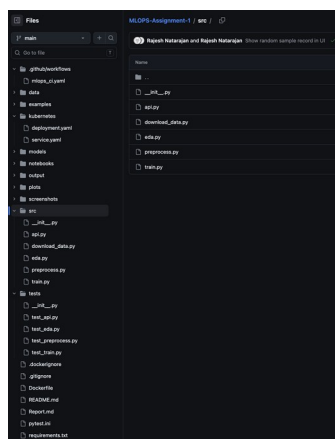


Figure 2. Repository folders and assignment files.

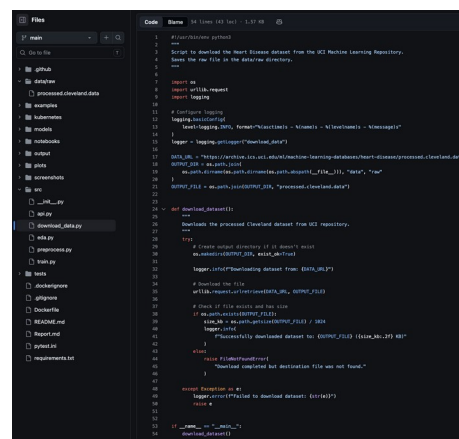


Figure 3. Reproducible data-download script.

2. Data acquisition and exploratory analysis

The download script retrieves the processed Cleveland dataset from the UCI repository and stores it as `data/raw/processed.cleveland.data`. It logs the source URL, destination, and file size so I can confirm that the same input was obtained in a fresh environment.

```

(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOPS Assignment 1 % ls -lh data/raw/processed.cleveland.data
python src/download_data.py
-rw-r--r-- 1 rajeshnatarajan staff 18K Jul 8 22:38 data/raw/processed.cleveland.data
2026-07-08 23:33:07,188 - download_data - INFO - Downloading dataset from: https://archive.ics.uci.edu/ml/machine-learning-d
atabases/heart-disease/processed.cleveland.data
2026-07-08 23:33:08,955 - download_data - INFO - Successfully downloaded dataset to: /Users/rajeshnatarajan/Documents/MLOPS
Assignment 1/data/raw/processed.cleveland.data (18.83 KB)
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOPS Assignment 1 %

```

Figure 4. Dataset downloaded successfully in the clean environment.

EDA findings

- The dataset contains 303 patient records and 13 input features.
- Question marks in `ca` and `thal` are converted to missing values before preprocessing.
- Target values 1 to 4 are mapped to one positive class; 0 remains the negative class.
- The binary classes are reasonably balanced: 164 negative and 139 positive records.

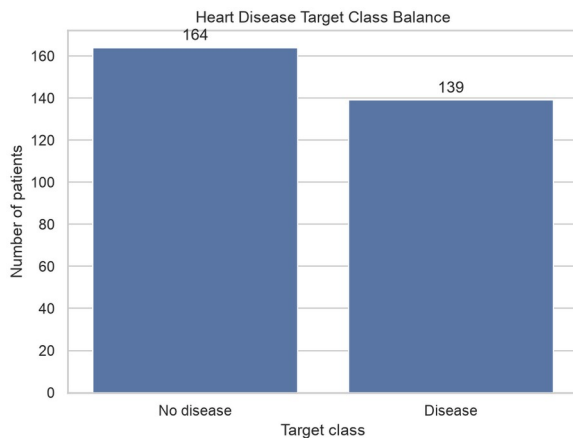


Figure 5. Class distribution after target mapping.



Figure 6. Missing-value count by feature.

The EDA script also generates numerical histograms, a correlation heatmap, and an age-versus-maximum-heart-rate plot. I kept these plots in code so they can be regenerated after a new download.

3. Preprocessing and modelling approach

I placed preprocessing and classification in one scikit-learn Pipeline. The fitted pipeline is saved as a single model artifact, which means the API applies the same transformations used during training.

Input group	Columns	Transformation
Numerical	age, trestbps, chol, thalach, oldpeak, ca	Median imputation and StandardScaler
Categorical	sex, cp, fbs, restecg, exang, slope, thal	Most-frequent imputation and OneHotEncoder
Target	num	0 stays 0; values 1-4 become 1

Modelling choices

I compared Logistic Regression as an interpretable baseline with Random Forest as a non-linear ensemble model. Both were tuned with five-fold GridSearchCV using ROC-AUC as the scoring metric.

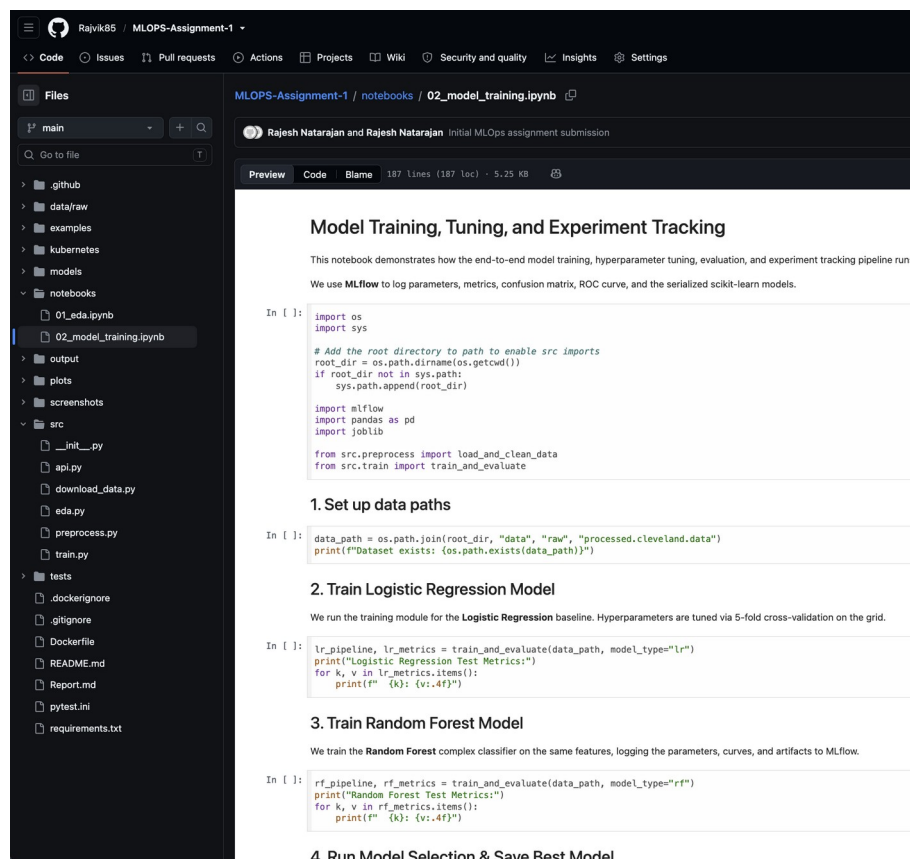


Figure 7. Training notebook showing data setup, model training, and model selection.

The notebook supports review and exploration, while `src/train.py` is the repeatable entry point used by automation.

4. Model evaluation and final selection

Model	Best settings	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	C=1.0, lbfgs	0.8689	0.8125	0.9286	0.8667	0.9578
Random Forest	depth=5, split=10, trees=100	0.8689	0.8333	0.8929	0.8621	0.9426

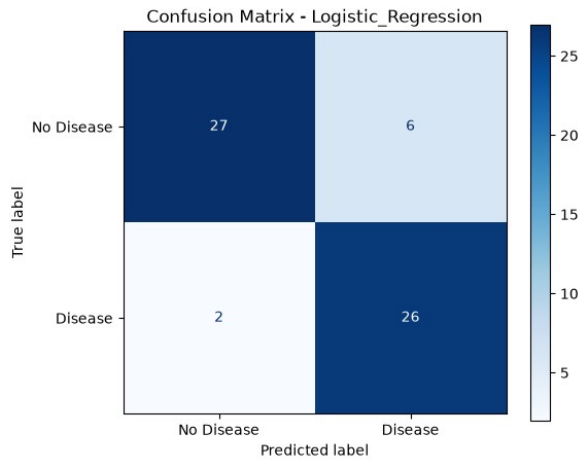


Figure 8. Logistic Regression confusion matrix.

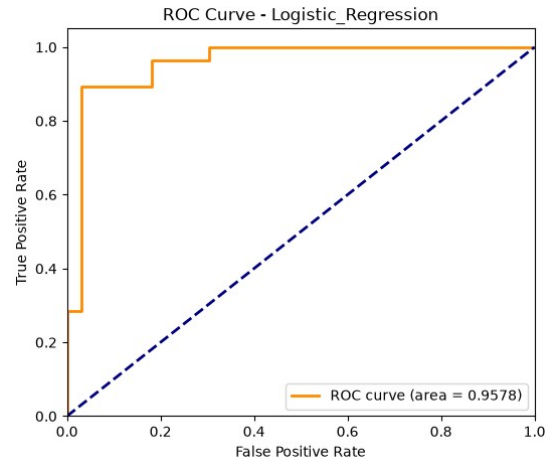


Figure 9. Logistic Regression ROC curve.

Selection decision

Both models produced the same test accuracy. Logistic Regression gave the stronger F1 score and ROC-AUC in this run, so I selected it as the final model. It is also easier to interpret and inexpensive to serve.

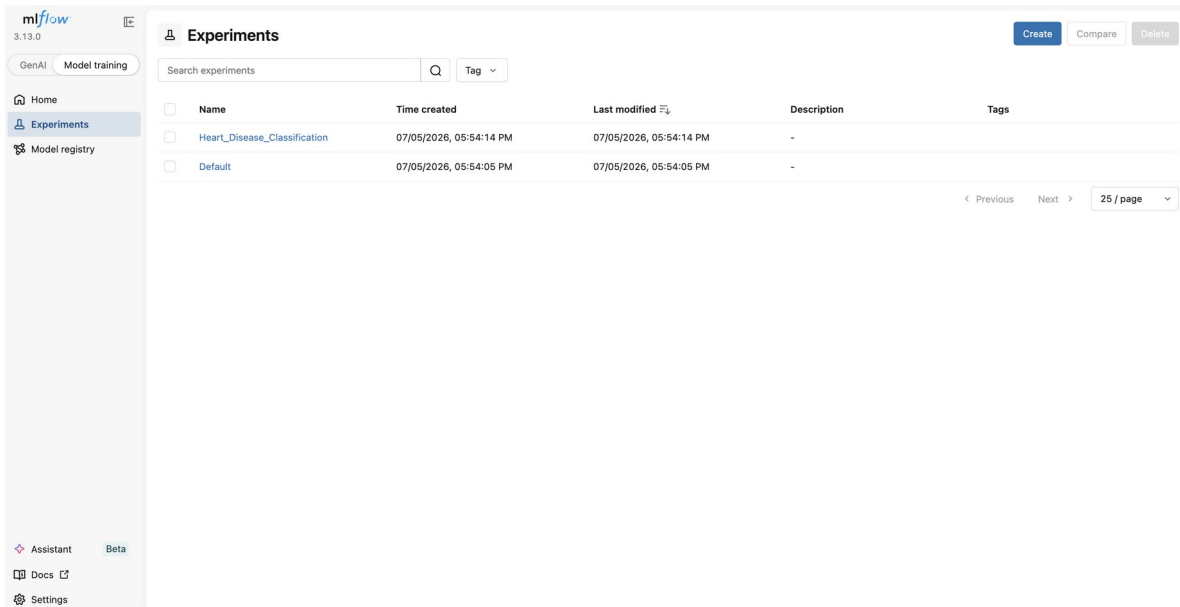
The saved models/best_model.joblib file contains both the fitted preprocessor and classifier. Saving them together reduces the chance of training-serving differences.

Evaluation measures used

- Accuracy records the overall proportion of correct predictions.
- Precision and recall show how the positive class is handled.
- F1 balances precision and recall.
- ROC-AUC compares ranking performance across classification thresholds.

5. Experiment tracking summary

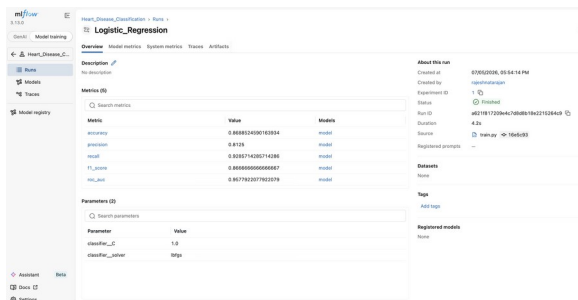
I logged both model runs under the Heart_Disease_Classification experiment in MLflow. Each run includes the chosen hyperparameters, evaluation metrics, confusion matrix, ROC curve, and fitted pipeline artifact.



The screenshot shows the MLflow Experiments page. On the left is a sidebar with navigation links: Home, Experiments (selected), Model registry, Assistant, Docs, and Settings. The main area is titled 'Experiments' and contains a search bar and a table of experiments. The table has columns for Name, Time created, Last modified, Description, and Tags. Two experiments are listed: 'Heart_Disease_Classification' and 'Default', both created on 07/05/2026 at 05:54:14 PM and 05:54:05 PM respectively. At the bottom right of the table, it says '25 / page'.

Name	Time created	Last modified	Description	Tags
Heart_Disease_Classification	07/05/2026, 05:54:14 PM	07/05/2026, 05:54:14 PM	-	
Default	07/05/2026, 05:54:05 PM	07/05/2026, 05:54:05 PM	-	

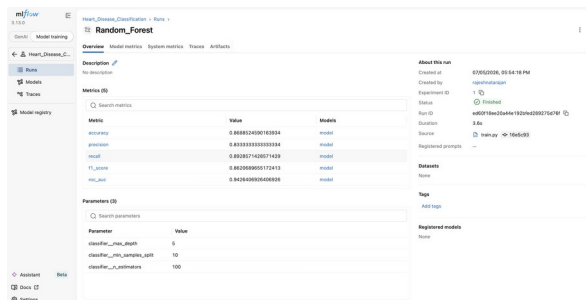
Figure 10. MLflow experiment list with completed training runs.



The screenshot shows the MLflow run overview for a 'Logistic_Regression' model. The left sidebar has links for Runs, Metrics, and Model registry. The main area is divided into several sections: 'Description' (no description), 'Metrics (3)' (a table with accuracy, precision, and recall), 'Parameters (2)' (a table with alpha and beta), 'About this run' (metadata like created at, created by, experiment ID, run ID, source, duration, and status), 'Datasets' (none), and 'Tags' (none). The 'Metrics' table shows accuracy: 0.86802409763934, precision: 0.878, and recall: 0.86802409763934. The 'Parameters' table shows alpha: 1.0 and beta: 0.0.

Parameter	Value
alpha	1.0
beta	0.0

Figure 11. Logistic Regression run overview.



The screenshot shows the MLflow run overview for a 'Random_Forest' model. The layout is similar to Figure 11. The 'Metrics' table shows accuracy: 0.86802409763934, precision: 0.86802409763934, and recall: 0.86802409763934. The 'Parameters' table shows alpha: 1.0 and beta: 100.

Parameter	Value
alpha	1.0
beta	100

Figure 12. Random Forest run overview.

The run pages are the source for the comparison metrics reported on page 5. The logged artifacts make it possible to review the selected settings and outputs without rerunning training.

6. API serving, browser UI, and monitoring

The FastAPI application loads the saved model when it starts. Pydantic validates the patient fields, and the prediction response returns the class, confidence, and model version.

Endpoint	Purpose	Expected result
GET /health	Readiness and model status	200 with model_loaded=true
GET /sample	Load a sample into the browser form	Valid patient payload
POST /predict	Run model inference	Prediction, confidence, model version
GET /metrics	Prometheus monitoring	Request counters and timing metrics

Figure 13. Browser prediction form with a completed result.

Figure 14. FastAPI documentation page and available endpoints.

```

$ curl http://127.0.0.1:8000/health
{"model_loaded": true}

$ curl http://127.0.0.1:8000/sample
{"age": 44, "sex": 1, "chest_pain_type": 2, "resting_bp": 130, "cholesterol": 219, "fasting_blood_sugar": 0, "rest_ecg": 2, "exercise_angina": 0, "slope": 1}

$ curl -X POST http://127.0.0.1:8000/predict
{"prediction": 0, "confidence": 8.33, "model_version": "1.0.0"}

$ curl http://127.0.0.1:8000/metrics
{"requests": {"total": 4, "successful": 4, "failed": 0}, "latency": {"min": 0.0001, "max": 0.0002, "avg": 0.00015}, "memory": {"total": 100, "used": 10}}

```

Figure 15. Local health, prediction, and metrics requests.

7. Automated testing and CI/CD workflow

The test suite covers the API, EDA output, preprocessing, and model-training utilities. In the clean environment shown below, Pytest collected 12 tests and all 12 passed with 66% overall coverage.

Test file	Main checks
tests/test_api.py	UI, sample, health, prediction, validation, and metrics
tests/test_preprocess.py	Missing values, target mapping, and fitted transformations
tests/test_eda.py	Required plot files are generated
tests/test_train.py	Training and model-selection utilities

```
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOPS Assignment 1 % python -m pytest tes
ti/ --covsrc --cov-report=term-missing
===== test session starts =====
platform darwin -- Python 3.13.5, pytest-9.1.1, pluggy-1.4.0
rootdir: /Users/rajeshnatarajan/Documents/MLOPS Assignment 1
configfile: pytest.ini
plugins: cov-7.1.0, anyio-4.14.1
collecting 0 items
collected 12 items

tests/test_api.py ..... [ 58%]
tests/test_eda.py ..... [ 66%]
tests/test_preprocess.py .. [ 83%]
tests/test_train.py .. [100%]

===== warnings summary =====
tests/test_api.py: 16 warnings
/tmp/mlops-clean-check/lib/python3.13/site-packages/joblib/numpy_pickle.py:207: DeprecationWarning: Se
tting the shape on a NumPy array has been deprecated in NumPy 2.0.
As an alternative, you can create a new view using np.reshape (with copy=False if needed).
array.shape = self.shape

tests/test_eda.py::test_generate_eda_plots creates required files
/tmp/mlops-clean-check/lib/python3.13/site-packages/seaborn/matrix.py:243: PendingDeprecationWarning:
The set_bad function will be deprecated in a future version. Use cmap.with_extremes(bad=...) or Colormap
(bad=...) instead.
self.cmap.set_bad(bad)

--- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform darwin, python 3.13.5-final-0

Name                Stmts   Miss  Cover   Missing
-----
src/_init_.py         0      0  100%
src/api.py            98     16    82%   175, 182-184, 278-285, 307-308, 334-336
src/download_data.py  22     22     0%   7-54
src/eda.py            80      0   100%   57-58, 120-134
src/preprocess.py     33      6    82%   44, 98-99, 106-108
src/train.py         118     65    45%   93-200, 235-278
TOTAL                343    117    66%

===== 12 passed, 17 warnings in 118.79s (0:01:58) =====
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOPS Assignment 1 %
```

Figure 16. Clean-environment Pytest and coverage result.

```
Y mlops_ci.yml M X
Users > rajeshnatarajan > Documents > MLOPS Assignment 1 > .github > workflows > Y mlops_ci.yml
9 jobs:
10 lint-and-test:
11 - name: Checkout Code
12
13
14
15
16 - name: Set up Python
17 uses: actions/setup-python@v5
18 with:
19   python-version: "3.13"
20
21
22 - name: Cache Pip Dependencies
23 uses: actions/cache@v4
24 with:
25   path: ~/.cache/pip
26   key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}
27   restore-keys: |
28     ${{ runner.os }}-pip-
29
30
31 - name: Install Dependencies
32 run: |
33   python -m pip install --upgrade pip
34   pip install -r requirements.txt
35
36
37 - name: Run Code Linting (Black & Flake8)
38 run: |
39   black --check src/ tests/
40   flake8 src/ tests/ --count --select=E9,F83,F7,F82 --show-source --statistics
41   flake8 src/ tests/ --count --max-line-length=127 --statistics
42
43
44 - name: Download Dataset
45 run: |
46   python src/download_data.py
47
48
49 - name: Run Unit Tests
50 run: |
51   pytest tests/ --cov=src --cov-report=term-missing
52
53
54 - name: Validate Training Pipeline
55 run: |
56   MLFLOW_ALLOW_FILE_STORE=true python -m src.train --model lr --save-path models/test_model.joblib
57
58
59 - name: Validate Docker Image Build
60 run: |
61   docker build -t heart-disease-api:test .
```

Figure 17. GitHub Actions workflow steps.

Failure and logging behavior

Black, Flake8, Pytest, training validation, and Docker build are separate workflow steps. If a command returns a non-zero status, GitHub Actions stops the job and shows the failed step and its command output.

8. Containerization and deployment workflow

The Dockerfile uses a two-stage Python 3.13 build. Dependencies are installed in the builder stage and copied into a smaller runtime stage. The API runs as the non-root appuser and includes a /health check.

```
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % docker build -t heart-disease-api-check .
[*] Building 3.6s (10/15) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/python:3.13-slim
=> [auth] library/python:3.13-slim token for registry-1.docker.io
=> [internal] load .dockerignore
=> [internal] load context: /
=> [builder 1/5] FROM docker.io/library/python:3.13-slimsha256:eb43ff125d8d8d7449dcb74336c23bca412f5
=> resolve docker.io/library/python:3.13-slimsha256:eb43ff125d8d8d7449dcb74336c23bca412f5
=> [internal] load build context
=> transferring context: 5.25kB
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [runner 3/5] RUN useradd -m -u 1808 appuser && mkdir -p /app/models && chown -
=> CACHED [builder 3/5] RUN apt-get update && apt-get install -y --no-install-recommends bui
=> CACHED [builder 4/5] COPY requirements.txt
=> CACHED [builder 5/5] RUN pip install --no-cache-dir --user -r requirements.txt
=> CACHED [runner 4/5] COPY --from=builder --chown=appuser:appuser /root/.local/home/appuser/.
=> CACHED [runner 5/5] COPY --chown=appuser:appuser src/ /app/src/
=> [runner 6/5] COPY --chown=appuser:appuser models/ /app/models/
=> exporting to image
=> exporting layers
=> exporting manifest sha256:b8259ddc78fde49c211f789cf4182ebdd67971ea3aee9d5a677d49bfe7ea
=> exporting config sha256:c21d184c07813a87376dbb0c76c2ba2c3d767c2af6a37b8b8b183ab776747f
=> exporting attestation manifest sha256:eb43ff125d8d8d7449dcb74336c23bca412f5
=> exporting manifest list sha256:b8259ddc78fde49c211f789cf4182ebdd67971ea3aee9d5a677d49bfe7ea
=> naming to docker.io/library/heart-disease-api-check
=> unpacking to docker.io/library/heart-disease-api-check
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/wdkzkg6envzyznpdvewj41r7
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 %
```

Figure 18. Docker image built in an isolated environment.

```
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % docker
ps --filter name=heart-disease-api-check
CONTAINER ID IMAGE COMMAND CREATED STATUS
9f0aac31fa97 heart-disease-api-check "uvicorn src.api:app-" 59 seconds ago Up 59 se
conds (healthy) 0.0.0.0:8766->8080/tcp, [::]:8766->8080/tcp heart-disease-api-check
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % curl ht
tp://127.0.0.1:8766/health
{"status":"healthy","model_loaded":true}
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % curl -
X POST "http://127.0.0.1:8766/predict" \
-H "Content-Type: application/json" \
--data @examples/patient_request.json
{"prediction":0,"confidence":0.19566462725688827,"model_version":"1.0.0"}
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % curl h
ttp://127.0.0.1:8766/health
{"status":"healthy","model_loaded":true}
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % curl -
X POST "http://127.0.0.1:8766/predict" \
-H "Content-Type: application/json" \
--data @examples/patient_request.json
{"prediction":0,"confidence":0.19566462725688827,"model_version":"1.0.0"}
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % docker
inspect --format '{{.State.Health.Status}}' heart-disease-api-check
healthy
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % ((((((
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 %
```

Figure 19. Healthy container and successful prediction.

Kubernetes deployment

- Two API replicas use a rolling-update strategy with maxUnavailable set to 0.
- Readiness and liveness probes call /health.
- CPU and memory requests and limits are defined.
- A LoadBalancer service maps port 80 to container port 8000.

```
(base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 % cd "/Users/rajes
hnatarajan/Documents/MLOps Assignment 1"

kubectl get pods
kubectl get svc heart-disease-api-service

curl http://127.0.0.1:18080/health

curl -X POST http://127.0.0.1:18080/predict \
-H "Content-Type: application/json" \
--data @examples/patient_request.json
{"prediction":0,"confidence":0.19566462725688827,"model_version":"1.0.0"}
(mlops-clean-check) (base) rajeshnatarajan@Rajeshs-MacBook-Pro MLOps Assignment 1 %
```

Figure 20. Kubernetes pods, service, health response, and prediction.

9. Requirement coverage and submission evidence

Production-readiness requirements

Requirement	Evidence included
Clean setup using requirements.txt	Setup commands on page 2; clean download and 12 passing tests on pages 3 and 8.
Model serves in an isolated environment	Docker build, healthy container, /health, and /predict evidence on page 9.
Pipeline fails clearly on errors	Separate Black, Flake8, Pytest, training, and Docker workflow steps on page 8.

Documentation and reporting compliance

Reporting requirement	Where it is covered
Setup/install instructions	Page 2, including environment, dependency, download, and test commands.
EDA and modelling choices	Pages 3-5, including findings, preprocessing, model comparison, and selection.
Experiment tracking summary	Page 6, including MLflow runs, metrics, and artifacts.
Architecture diagram	Page 2, Figure 1.
CI/CD and deployment screenshots	Pages 8-9, Figures 16-20.
Link to code repository	Cover page and the repository link below.

Code repository: <https://github.com/Rajvik85/MLOPS-Assignment-1>

Local API access

```
docker build -t heart-disease-api:check .
docker run -d --name heart-disease-api-check -p 8766:8000 heart-disease-api:check
open http://127.0.0.1:8766
```

The API is intended for local assessment unless a public URL is supplied separately. The screenshots in this report show the commands and responses I used to verify the completed workflow.